The meaning of the interface Segregation Principle is

- a. Classes should depend on as many interfaces as possible
- b. Classes should depend only on the interfaces they need ✔
- c. Interfaces should include as many features as possible
- d. Interfaces should include only cohesive features (i.e.needed together) ✔
- e. None of the above

The meaning of the Open-Closed Principle is

- a. A class should be inheritable ✖
- b. A module should depend on abstractions that are instantiated depending on the specific context
- c. A functional change can be done by modifying the existing code
- d. A functional change should not affect the existing code ✔
- e. None of the above

The meaning of the Liskov Substitution Principle is:

- a. An object of the superclass should always be replaceable by an object of the subclass ✔
- b. An object of the subclass should always be replaceable by an object of the superclass
- c. A subclass can do less than the superclass
- d. A subclass should do at least what the superclass does ✔
- e. A subclass can do something completely different than the superclass

The meaning of the Dependency Inversion Principle includes:

- a. Specific/Concrete classes should depend on Abstractions
- b. High level layers should depend on the implementations in the low level layers
- c. High level layers should depend on the interfaces that are implemented in the low level layers ✔
- d. Low level layers depend on the high level layers
- e. None of the above

The meaning of the Single Responsibility Principle is:

- a. A class should have just one method
- b. A class should do just one thing
- c. A module should have just one reason to change ✔
- d. A module should be responsible to just one user/stakeholder/actor ✔
- e. All of the above

The dependency structure of the packages of a release component should represent a

- (•) a. Directed Acyclic Graph
- ( ) b. List
- ( ) c. Binary tree
- ( ) d. Directed Cyclic Graph
- ( ) e. Tree

Which of the following are good design principles

- [ ] a. High coupling
- [x] b. Low coupling
- [x] c. High cohesion
- [ ] d. Low cohesion
- [ ] e. None of the above

Which of the following are good criteria for grouping classes together into packages

- [x] a. All classes in a package should be reusable
- [x] b. All classes in a package should change for the same reason(s)
- [ ] c. All classes in a package should be in an inheritance relation
- [x] d. All classes in a package should be reused together
- [ ] e. All classes in a package should implement the same interfaces

Abstract packages

- [x] a. Should contain abstract classes
- [x] b. Should be depended upon
- [ ] c. Should depend on other packages
  Binary tree
- [ ] d. Doesn't matter if they are stable or not
- [x] e. Should be stable

The stability of a package in a design

- [x] a. Should be as low as possible
- [ ] b. Does not depend on the other packages
- [x] c. Depends both on the incoming and outgoing dependencies
- [ ] d. Should to be as high as possible
- [ ] e. Is not that important as long as the package depends on more stable ones

Which of the following statements is true?

- [x] a. The ViewModel represents a Model for the View
- [x] b. MVVM is a variant of the MVC pattern
- [x] c. Views can be designed independent from the code behind in MVVM
- [x] d. Several Views can have references to the same ViewModel
- [x] e. The ViewModel is a Façade to the Model

Which statements are true about MVC:

- [ ] a. Cannot be applied in a layered architecture
- [ ] b. It is highly decoupled
- [x] c. The View and the Controller depend on the Model
- [ ] d. The Model depends on the View and the Controller
- [x] e. It is an instance of the Observer DP

Suitable solution(s) for a decoupled, distributed architecture of heterogenous, independent components/services is(are):

- [x] a. Peer-to-peer
- [ ] b. Client-Server
- [ ] c. 3-tiered
- [ ] d. MVC
- [x] e. Broker

The Broker component in Broker pattern:

- [x] a. Returns the most suitable server based on a client request
- [x] b. Represents a single point of failure
- [ ] c. Holds both a collection of Clients and a collection of Servers
- [ ] d. Holds a collection of Clients
- [x] e. Holds a collection of Servers

Which of the following are good practices for a layered architecture

- [x] a. Changes made in one layer of the architecture should not impact components in other layers
- [ ] b. A layer may depend on all the lower layers
- [ ] c. A layer may depend only on the immediate lower layer
- [x] d. A layer should depend on interfaces implemented by the lower layers
- [ ] e. None of the above

Considering Service design principles we can state that

- [x] a. Services are loosely coupled ✔
- [ ] b. Services should maintain state information
- [x] c. Services should be discoverable no matter where they actually reside ✔
- [ ] d. We should know the underlying business logic in order to use services
- [x] e. We should be able to reuse services in various business flows ✔

What is true about SOAP based services:

- [ ] a. They are based on standards such as XML
- [ ] b. The interface offered by the service is described by using the Web Services Description Language
- [ ] c. The architecture is an instance of the Broker pattern
- [ ] d. SOAP is a protocol
- [x] e. All of the above ✔

REST services are different from SOAP-based services because:

- [x] a. REST is a set of architectural principles ✔
- [x] b. A response to a REST request can be in several formats (i.e. XML, HTML, plain text, JSON) ✔
- [x] c. REST services are stateless ✔
- [ ] d. REST does offer built-in security and transaction management
- [ ] e. REST reads cannot be cached

In the context of Domain driven design the following statements hold:

- [x] a. Value Objects are not tracked while Entities should be trackable ✔
- [x] b. Value Objects represent attributes without identity ✔
- [ ] c. If the Value Objects are shared, they should be mutable
- [ ] d. Value Objects cannot contain other Value Objects
- [x] e. Entities represent uniquely identifiable concepts ✔

Aggregates in the context of DDD

- [x] a. The root entity has global identity while the inner entities have local identity ✔
- [ ] b. The aggregated objects are also accessible from outside
- [ ] c. The aggregated objects can exist even if the root is deleted
- [x] d. Group together objects that are a unit related to data change ✔
- [ ] e. May have several roots

In the iDesign method:

- [ ] a. The iDesign Architecture is a strict Layered one
- [x] b. A Manager service addresses the sequence volatility
- [ ] c. There is a 1-to-1 relationship between Manager and Engine services
- [ ] d. A Manager service encapsulates a set of unrelated use-cases
- [x] e. Engine services encapsulate business rules and activities

Some best practices in the iDesign method are:

- [x] a. Clients do not call Engines
- [ ] b. Clients do publish events
- [ ] c. Engines may receive queued calls
- [ ] d. Engines may call each other
- [x] e. Managers do not queue calls to more than one Manager in the same use-case

Very small services(microservices) have consequences such as:

- [ ] a. Enable better performance
- [ ] b. Are not appropriate for independent database updates
- [x] c. Are usually self-sufficient
- [x] d. Are not appropriate for transactional database updates
- [x] e. Decrease performance if many services need to cooperate

The following is true about orchestration and choreography

- [x] a. Orchestration involves a coordinating service that handles the microservices interaction
- [x] b. The orchestrator is a single point of failure in a process
- [x] c. Choreography is a decentralized, event-driven approach
- [x] d. Choreography allows for removing or adding services anytime from the event stream
- [ ] e. None of the above

The following is true in the iDesign method:

- [ ] a. It is the responsibility of the Engine component to translate business transactions into CRUD operations
- [ ] b. Utility Services are called only by Manager components
- [ ] c. A Resource can be just a database
- [x] d. A ResourceAccess service encapsulates the volatility in accessing a Resource
- [x] e. It is the responsibility of the ResourceAccess component to translate business transactions into CRUD operations

Active Record violates the following principle(s):

- [x] a. Single responsibility
- [ ] b. Dependency inversion
- [ ] c. Open-closed
- [ ] d. Interface segreggation
- [ ] e. Liskov Substitution

The following is true about the Data Mapper:

- [x] a. Its responsibilities are to retrieve data from the storage and to map it on the business logic structures
- [x] b. Adds a layer that decouples the Business logic from the Data Access
- [ ] c. Its responsibilities are to map the Business logic structures on the presentation
- [ ] d. Should be used with Transaction Script
- [x] e. Can be used with Table Data Gateway

Transaction script is an example of business logic design based on

- [x] a. Use-case driven decomposition
- [x] b. Functional driven decomposition
- [x] c. Transaction driven decomposition
- [ ] d. Domain driven decomposition
- [ ] e. Volatility driven decomposition

Considering how inheritance is reflected in a relational database structure the following is true:

- [ ] a. There is no way to reflect inheritance in a relational database structure
- [x] b. A table can contain the objects of all the classes in an inheritance tree
- [x] c. A table can contain the objects of any class in an inheritance tree
- [x] d. A table can contain the objects of a concrete class in the inheritance tree
- [ ] e. Inheritance can be applied to tables the same way as it is applied to classes

The responsibilities of the Data access components are

- [ ] a. Map the data structure to the domain model structure
- [x] b. Store the raw data
- [x] c. Execute queries
- [x] d. Connect to the database
- [ ] e. Contain some business logic

The Transform View

- [ ] a. Embeds programing logic into a HTML page
- [x] b. Acts like a mapper between the XML model and the HTML to be displayed
- [x] c. Enables portability
- [x] d. Makes it easy to change the appearance of a web page
- [ ] e. Builds the structure of the HTML page based on a template

The following is true about concurrency control

- [ ] a. Optimistic concurrency control relies on hope that conflicts will not happen
- [x] b. Optimistic concurrency control detects conflicts
- [ ] c. Isolated or immutable data needs concurrency control
- [x] d. Pessimistic concurrency control prevents conflicts
- [ ] e. Conflicts cannot be avoided

A Page Controller

- [x] a. Decodes the URL and extracts the data for action
- [ ] b. Uses Command objects to delegate actions
- [x] c. Forwards the data to be displayed to the appropriate view
- [x] d. Invokes model objects to process the data
- [ ] e. Handles duplicated features

The Unit of Work pattern is used to

- [x] a. Maintain the consistency of the database
- [x] b. Enhance the performance by avoiding several small database updates
- [x] c. Maintain the changes of all objects affected by a business transaction
- [ ] d. Eliminate deadlocks
- [ ] e. Maintain all the transactions involved in a use-case

A transaction has the following properties

- [ ] a. Dependency Inversion (i.e. the transaction should not depend on other transactions)
- [ ] b. Interface seggregation (i.e. the transaction needs to have a well defined interface)
- [x] c. Isolation (i.e. the transaction cannot be interrupted)
- [x] d. Consistency (i.e. the system should be in a consistent state before and after the transaction)
- [x] e. Atomicity (i.e. the sequence of operations cannot be split further into the individual operations)

The Abstract Factory is used to

- ○ a. Create abstract classes
- ◉ b. Create objects of different classes that need to be compatible
- ○ c. Create prototypes
- ○ d. Create interfaces
- ○ e. None of the above

A factory method

- ☐ a. Cannot use a prototype to create the product
- ☑ b. Promotes subclassing to change the product that gets created
- ☐ c. Is always defined with the "creator" class
- ☑ d. Encapsulates the creation of a specific product
- ☑ e. Can be parameterized to avoid subclassing

A prototype object

- ☑ a. Enables decoupling between the Creator and the Product
- ☑ b. Represents the Product that needs to be instantiated
- ☐ c. Uses Deep copy to implement the clone operation
- ☑ d. Can be specified at run-time
- ☑ e. Contains a clone method to create other similar objects

To create a class that allows to be instantiated in a fixed number of objects we have to

- ☐ a. Use a prototype
- ☐ b. Make the constructor public
- ☑ c. Define a public method to access the created instances
- ☑ d. Make the constructor private
- ☑ e. Keep the created instances

An Abstract Factory can contain

- ○ a. A set of factory methods
- ○ b. A set of prototype objects
- ○ c. A parameterized factory method
- ○ d. Both factory methods and prototypes
- ◉ e. All of the above

What strategies is the Bridge Design pattern applying:

- [x] a. Encapsulate the volatilities ✔
- [x] b. Favor composition over inheritance ✔
- [x] c. Depend on abstractions instead of specific classes ✔
- [x] d. Be open for extension and closed for modification ✔
- [x] e. Concern separation ✔

The following is true about the Composite pattern

- [x] a. Both the primitive elements (leaves) and the containers (composites) need to respect the same interface ✔
- [x] b. Composites may contain both leaves and composites ✔
- [x] c. The container operations should always be placed in the Component interface ✖
- [x] d. Composites may contain only leaves ✖
- [x] e. The components should never have a reference to their container (composite) ✖

If we want to define a standard car and several options that could be added to build a custom car we would use the following design pattern:

- [x] a. Factory method ✖
- [x] b. Proxy ✖
- [x] c. Decorator ✔
- [x] d. Bridge ✖
- [x] e. Adapter ✖

From the Clients perspective, the objective of the Adapter Design pattern is to:

- ( ) a. Encapsulate how objects used by the Client are created
- ( ) b. Decouple the Target from the Adaptee
- ( ) c. Override the behavior of the Adaptee
- (•) d. Allow using objects that are not compliant with the Target interface ✔
- ( ) e. Provide a uniform interface that the Adaptees need to implement

A site displaying blurred, low-resolution copy of the images instead of the high-resolution is an example of using the following design pattern:

- [x] a. Proxy ✔
- [x] b. Bridge ✖
- [x] c. Adapter ✖
- [x] d. Decorator ✖
- [x] e. Factory method ✖

Which DP(s) is/are more suitable to model the concept of a thread considering its lifecycle that includes the following possible states: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED

- [x] a. State
- [ ] b. Decorator
- [ ] c. Adapter
- [ ] d. Strategy
- [ ] e. Composite

The following is an example of applying the Observer DP:

- [x] a. MVC
- [ ] b. Data Access Objects
- [x] c. Listeners in Java
- [ ] d. Broker
- [ ] e. Data Mappers

Which DP(s) is/are more suitable to model the transportation from the hotel to the airport considering the existing options: by train, bus or taxi.

- [ ] a. Composite
- [ ] b. Decorator
- [ ] c. State
- [x] d. Strategy
- [ ] e. Adapter

Which DP(s) is/are more suitable to model the concept of a stock trader(agent de bursa) that gets buying or selling orders from a client

- [ ] a. Composite
- [ ] b. Bridge
- [ ] c. Strategy
- [x] d. Command
- [x] e. Proxy

Considering the Chain of responsibility DP which of the following is true?

- [ ] a. It is similar to the Observer DP because the sender holds a collection of receivers
- [x] b. It is different from the Command DP because it may have several request handlers
- [ ] c. It is different from the Decorator DP because it involves sequence of linked objects
- [ ] d. It is similar to the Decorator DP because all the linked objects have the same interface
- [x] e. It is different from the Command DP because it the request is not modeled as an object